1. Enunciado

En este proyecto se pretende construir un sistema de gestión de trabajo colaborativo a través de tableros de tareas. Como referencia para identificar las características de este tipo de aplicaciones se pueden explorar aplicaciones como Trello (https://www.trello.com/).

La aplicación deberá permitir:

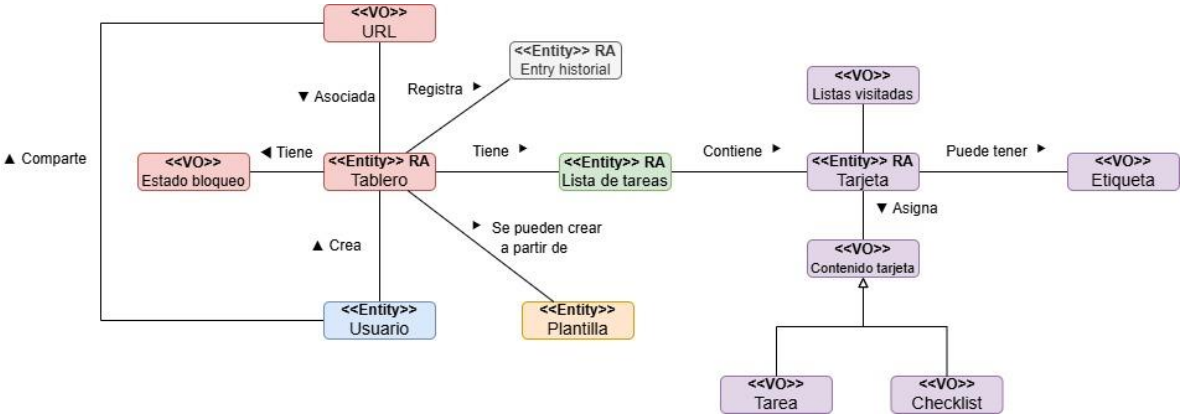
- Crear y modificar *tableros* donde hay *listas de tareas*. Una lista de tareas tiene *tarjetas* de manera que las tarjetas sirven para asignar tareas o para anotar información relevante.
- Las tarjetas se pueden marcar como completadas, en cuyo caso pueden pasar a una lista especial del tablero de tarjetas completadas.
- Las tarjetas pueden tener etiquetas para clasificarlas. Una etiqueta además tiene un color.
- Hay dos tipos de tarjetas: tarjetas que tienen tareas y tarjetas que tienen checklists.
- Los tableros mantienen una historia de todas las acciones de los usuarios. Por ejemplo, si se mueve una tarjeta entre listas de tareas se debe registrar una traza de que este movimiento se ha realizado.
- Es posible bloquear temporalmente un tablero para que no se puedan añadir nuevas tarjetas, solo mover tarjetas entre sus listas (por ejemplo, un tablero con TODO, DOING, DONE en el que durante una semana no se pueden añadir nuevas tarjetas).
- Para simplificar la gestión de los usuarios se utilizará una estrategia simple: cualquier persona con un correo electrónico podrá crear un tablero y obtendrá una URL única (y privada a no ser que la comparta) para su tablero.
- Un tablero puede compartirse con otros usuarios de la aplicación, para ello se comparte la URL.
- Implementar reglas a nivel de tablero o de lista de tareas:
    - Una lista no puede tener más de N ítems (configurable)
    - Una lista define que una tarjeta tiene que haber pasado por otras listas antes de llegar a ella.
- Filtrado de tarjetas por etiquetas.
- Creación de plantillas para tableros.
    - Las plantillas se definen como un fichero YAML.

- Autenticación basada en código por correo.

2. Lenguaje ubicuo

Término	Descripción
Tablero	Espacio de trabajo en el que colocar listas de tareas
Tarea	Una acción del proyecto que puede ser asignada a una tarjeta
Lista de tareas	Lista que contiene tarjetas
Tarjeta	Componente de una lista de tareas que almacena una tarea o anota información relevante
Etiqueta	Cada etiqueta es de un color y sirve para clasificar tarjetas
Checklist	Conjunto de tareas que, una vez completo, completan la tarjeta
TODO	Representa el estado de una tarea pendiente
DOING	Representa el estado de una tarea en proceso
DONE	Representa el estado de una tarea completada
Plantilla	Fichero que permite copiar la estructura y reglas de un tablero para construir uno nuevo
Historial	Registro de actividades en el tablero
Entrada historial	Cada una de los eventos registrados en el historial

3. Modelo conceptual



4. Historias de usuario

4.1. Épica A → Gestión de tableros

HA1 – Crear tablero

Como usuario con email
Quiero crear un tablero nuevo
Para disponer de un espacio con listas donde gestionar tareas

Criterios de aceptación

* Al crear un tablero, se genera una URL única para el tablero
* Se puede inicializar el tablero vacío o desde una plantilla YAML
* Devuelve ID del tablero y URL

HA2 – Bloquear tablero

Como usuario con acceso al tablero
Quiero configurar un bloqueo temporal del tablero
Para no poder añadir nuevas tarjetas y solo moverlas entre las listas del tablero

Criterios de aceptación

* Puedo activar/desactivar bloqueo con rango de fechas (desde - hasta)
* Los cambios quedan registrados creando una Entry Historial

HA3 – Desbloquear tablero

Como usuario con acceso al tablero
Quiero desactivar el bloqueo del tablero
Para trabajar con la configuración de bloqueo libremente

Criterios de aceptación

* El tablero deja de estar bloqueado instantáneamente, y no espera hasta la fecha final del bloqueo establecido
* Se permite añadir tarjetas libremente mientras que no se incumplan el resto de invariantes independientes al bloqueo como el límite N
* El desbloqueo queda registrado creando una Entry Historial

HA4 – Compartir tablero

Como usuario con acceso al tablero
Quiero compartir la URL del tablero con otros usuarios
Para que puedan acceder y trabajar sobre el mismo tablero

Criterios de aceptación

* Quien tenga la URL puede acceder y editar el tablero
* La URL es única para cada tablero

HA5 – Modificar nombre del tablero

Como usuario con acceso al tablero
Quiero modificar el nombre del tablero
Para tener una mejor gestión de los diferentes espacios de trabajo

Criterios de aceptación

* El tablero adquiere el nuevo nombre introducido
* La URL no cambia ni los usuarios pierden acceso a pesar de haber cambiado de nombre
* La modificación queda registrada con una Entry Historial
* No se permite un nombre vacío o con caracteres inválidos y devuelve error de validación en ese caso

HA6 – Eliminar tablero

Como usuario con acceso al tablero
Quiero eliminar completamente un tablero
Para deshacerme de espacios de trabajo innecesarios

Criterios de aceptación

* El tablero desaparece de entre los tableros disponibles
* La URL asociada ya no da acceso al tablero
* El historial del tablero desaparece y ya no es accesible

HA7 – Configurar prerrequisitos número de tarjetas por lista de tablero

Como usuario con acceso al tablero
Quiero definir que las listas no puedan contener más de N tarjetas (configurable)
Para evitar sobrecargar el flujo de tareas y trabajo

Criterios de aceptación

* Si N se alcanza, añadir una tarjeta a la lista falla
* Si una tarjeta se va fuera de la lista y queda hueco, se puede añadir otra
* El límite puede modificarse y se registra la modificación en el historial
* La lista especial no se configura con límite N

4.2. Épica B → Listas y reglas por lista

HB1 – Crear/editar/eliminar listas

Como usuario con acceso al tablero
Quiero crear, renombrar y eliminar listas dentro del tablero
Para organizar el flujo de tareas y trabajo

Criterios de aceptación

* Las listas tienen un nombre único en el tablero
* La creación, modificación y eliminación de listas crean una nueva Entry Historial
* Borrar una lista borrará todo el contenido dentro de esa lista

HB2 – Configurar límite de N ítems en una lista concreta

Como usuario con acceso al tablero
Quiero poner un límite de tarjetas N (configurable) en una lista
Para evitar sobrecargar el flujo de tareas y trabajo

Criterios de aceptación

* Si N se alcanza, añadir una tarjeta a la lista falla
* Si una tarjeta se va fuera de la lista y queda hueco, se puede añadir otra
* El límite puede modificarse y se registra la modificación con una Entry Historial
* Intentar configurar límite N en una lista especial falla

HB3 – Configurar prerrequisitos de lista

Como usuario con acceso al tablero
Quiero definir que para entrar en la lista C una tarjeta debe haber pasado por listas A y B antes
Para imponer orden en el flujo de trabajo

Criterios de aceptación

* Intentar mover una tarjeta a lista C sin haber pasado por las listas requeridas falla
* La lista de visitados de la tarjeta mantiene las listas por las que ha pasado la tarjeta

HB4 – Crear lista especial

Como usuario con acceso al tablero
Quiero definir que una lista es especial de tarjetas completadas
Para reflejar un flujo ordenado y organizado de trabajos completados

Criterios de aceptación

* Si ya hay una lista especial creada anteriormente, la creación fallará
* Las tarjetas marcadas como completadas se mueven a esta lista
* El criterio a nivel de tablero de límite N no afecta a esta lista
* Se crea una nueva Entry Historial que indique la configuración de la lista especial

4.3. Épica C → Tarjetas

HC1 – Crear tarjeta (tarea/checklist)

Como usuario con acceso al tablero
Quiero añadir una tarjeta a una lista con contenido (tarea o checklist)
Para capturar un trabajo o información relevante

Criterios de aceptación

* Se puede crear la tarjeta como una tarea o como una checklist que contenga una lista de ítems
* Si el tablero está bloqueado para añadir, la operación falla
* La tarjeta obtiene un ID único y la creación provoca una nueva Entry Historial

HC2 – Mover tarjeta entre listas

Como usuario con acceso al tablero
Quiero mover tarjetas entre listas
Para reflejar el progreso del trabajo

Criterios de aceptación

* Los movimientos válidos se permiten, a no ser que incumpla prerrequisitos de la lista como el límite N
* Cada movimiento genera una nueva Entry Historial
* Si el tablero está bloqueado, mover tarjetas está permitido

HC3 – Marcar tarjeta como completada

Como usuario con acceso al tablero
Quiero marcar una tarjeta como completada
Para que pase a una lista concreta especificada

Criterios de aceptación

* La acción mueve la tarjeta a la lista especial
* Si en una checklist todos los ítems han sido marcados como completados, la tarjeta se completa automáticamente
* Si la tarjeta no ha pasado por las listas necesarias antes de pasar a la lista especial, la operación falla
* Si no existe lista especial de completadas, la operación falla y se lanza un error de dominio indicando que no hay lista especial especificada

HC4 – Añadir/eliminar/modificar etiquetas en tarjeta

Como usuario con acceso al tablero
Quiero añadir etiquetas (color + nombre) a las tarjetas
Para poder filtrarlas y clasificarlas

Criterios de aceptación

* La etiqueta se añade individualmente a cada tarjeta
* Se pueden añadir múltiples etiquetas a una tarjeta
* Filtrado por etiqueta devuelve tarjetas que contengan esa etiqueta
* La creación, eliminación o modificación de las etiquetas se ve reflejada en una Entry Historial

HC5 – Editar contenido de la tarjeta

Como usuario con acceso al tablero
Quiero editar la descripción de la tarea o modificar la checklist de ítems
Para mantener la información actualizada

Criterios de aceptación

* Se reemplaza el contenido de la tarjeta
* Cada edición genera una nueva Entry Historial

4.4. Épica D → Plantillas

HD1 – Definir plantilla en YAML

Como usuario con acceso al tablero
Quiero crear una plantilla (fichero YAML) que describa listas iniciales y sus reglas
Para reutilizar configuraciones de tablero

Criterios de aceptación

* La plantilla incluye un nombre, las mismas listas y reglas por lista que en el tablero que se ha creado la plantilla
* El YAML se valida, error si no cumple el formato

HD2 – Crear tablero desde plantilla

Como usuario
Quiero crear un tablero a partir de una plantilla YAML
Para iniciar rápidamente la estructura con reglas

Criterios de aceptación

* El tablero se crea con listas y reglas tal y como define la plantilla
* Con una nueva Entry Historial se registra que el tablero se ha creado usando la plantilla

4.5. Épica E → Autenticación y seguridad

HE1 – Login por código email

Como usuario con correo
Quiero iniciar sesión introduciendo mi email y un código que me llega por correo
Para no gestionar contraseñas

Criterios de aceptación

* La petición de login genera un código temporal enviado por email
* El código expira pasado cierto tiempo

4.6. Épica F → Historial

HF1 – Registrar eventos en el historial del tablero

Como usuario con acceso al tablero
Quiero registrar en un historial las acciones sobre el tablero
Para monitorizar cambios y ver quién ha hecho cada acción

Criterios de aceptación

* Cada acción generará una Entry Historial de un tipo concreto.
* Cada registro lleva un timestamp, TableroId y los detalles del tipo de registro. Formato del registro ejemplo: EntryId, TableroId, tipo, usuario (email), timestamp, detalles (e.g. TarjetaMovida tarjetaId fromListId toListId)

HF2 – Consultar historial de un tablero

Como usuario con acceso al tablero
Quiero consultar el historial de acciones del tablero
Para saber qué cambios se han realizado, cuándo y quién los ha hecho

Criterios de aceptación

* Se consulta correctamente el historial asociado a un TableroId
* El resultado es paginado, conteniendo un máximo de 20 entradas por página
* En cada entrada se observa EntryId, tipo, usuario (email), timestamp y detalles (e.g. TarjetaMovida tarjetaId fromListId toListId)

4.7. Épica G → Búsqueda y filtrado

HG1 – Filtrar tarjetas por etiqueta

Como usuario con acceso al tablero
Quiero ver solamente tarjetas que contienen X etiquetas
Para centrarme en un subconjunto de trabajo

Criterios de aceptación

* Se puede filtrar por una o varias etiquetas a la vez (AND/OR configurable) especificando únicamente el nombre de la etiqueta
* No aparecen tarjetas que no cumplan con los filtros de la búsqueda
* Si no se configura AND/OR, por defecto se filtrarán con modo OR
* Si no se indican etiquetas se devuelve un error de validación

4.8. Épica H → Reglas de negocio y validaciones (historias técnicas)

HH1 – Validar límite N al añadir/mover tarjetas

Como sistema
Quiero impedir acciones que violen el límite N de una lista
Para mantener el invariante de capacidad

Criterios de aceptación

* Las operaciones de añadir tarjeta y mover tarjeta comprueban el límite antes de aplicarse
* Si se incumple la regla, se lanza una excepción de dominio

HH2 – Validar prerrequisitos de listas al mover tarjeta

Como sistema
Quiero comprobar las listas visitadas de la tarjeta
Para validar prerrequisitos y garantizar el flujo definido

Criterios de aceptación

* La operación de mover tarjeta comprueba las listas ya recorridas por esa misma tarjeta
* Si no se cumple, la operación se rechaza con un mensaje que indica qué listas faltan

5. Reglas de negocio

- R1 – Cuando un tablero está bloqueado, durante ese tiempo no se pueden añadir nuevas tarjetas, solo moverlas entre sus listas.

- R2 – Cuando una lista está configurada con límite de ítems N, no se pueden añadir ni mover nuevas tarjetas a esta lista.
- R3 – Si una lista define que una tarjeta tiene que haber pasado por otras listas antes de llegar a ella, solo aceptará tarjetas que cumplan dicho prerrequisito.
- R4 – Si existe lista especial, marcar tarjeta como completada mueve la tarjeta a esa lista. Si no existe, la operación falla y se lanza un error de dominio indicando que no hay lista especial completada.
- R5 – Toda acción que modifique el estado de un tablero, lista o tarjeta debe generar exactamente una Entry Historial que contenga: EntryId, TableroId, tipo, usuario (email), timestamp y detalles según el tipo de Entry.
- R6 – Una Entry Historial es append-only y no se modifica, es inmutable. Solo se elimina en cascada cuando se elimina el tablero (hard delete).
- R7 – El historial consultado para un tablero es el conjunto ordenado de manera descendente por timestamp de Entry Historial con el mismo TableroId. Este historial está paginado.
- R8 – El sistema permite obtener las tarjetas de un tablero filtrando por un conjunto de etiquetas aplicando uno de los dos modos: OR (una tarjeta se obtiene si contiene al menos una etiqueta del filtro) o AND (una tarjeta se obtiene si contiene todas las etiquetas del filtro). La comparación de etiquetas se hace solo por nombre.
- R9 – Cuando una tarjeta se crea, debe de crearse dentro de una lista y se añadirá en la última posición de la lista.

6. Entidades y objetos valor

Definimos entidades a aquellos componentes del dominio que tienen identidad propia y que pueden mutar a lo largo de su ciclo de vida. Por otro lado, el value object se define por sus atributos y no tiene identidad propia, comúnmente tratándose como objetos inmutables y sustituyéndolos por otros nuevos en caso de ser modificados. Con esta definición, categorizamos cada elemento de nuestro dominio:

- **Tablero:** Entidad, puesto que tiene identidad propia y puede ir cambiando a lo largo de su ciclo de vida. Su nombre puede cambiar sin perder identidad y la URL nunca cambiará aunque se modifique el nombre.
- **Lista de tareas:** Entidad, ya que también tiene identidad propia y cada lista puede ser modificada. Tiene reglas propias asociadas y necesitamos distinguir entre unas listas y otras.

- **Tarjeta:** Entidad, ya que puede haber dos tarjetas con el mismo contenido y etiquetas y aún así ser distintas, tiene identidad.
- **Usuario:** Entidad por definición, tienen identidad, se identifican por el email.
- **Plantilla:** Entidad, porque cada plantilla es diferente, ya que se crea mediante un fichero YAML que copia la estructura de cada entidad Tablero.
- **URL:** Value object, no tiene identidad propia en el dominio, solo importa el valor del enlace en sí.
- **Estado bloqueo:** Value object, simplemente importan sus atributos de saber si está activo y la fecha (desde cuándo hasta cuándo).
- **Entry historial:** Entidad, cada entrada tiene una identidad única y diferente. Las entradas que coinciden en el ID del tablero formarán el historial completo de dicho tablero. Cada entrada es de forma append-only, el historial está paginado y es accesible desde la interfaz de usuario. Aunque sea inmutable se sigue considerando entidad.
- **Listas visitadas:** Value object, no tiene identidad, es solamente un conjunto de valores inmutable que se utiliza para validar prerrequisitos.
- **Etiqueta:** Value object, importa solamente por su nombre y su color, son los atributos que observamos, no necesita de identidad propia. En el caso de considerar un catálogo de etiquetas disponible y gestionable, sí se podría tratar como una entidad, pero no lo modelaremos así.
- **Contenido tarjeta:** Value object, es simplemente el contenido dentro de la tarjeta, puede ser igual en varias tarjetas. No necesita identidad.
- **Tarea:** Value object, es un tipo concreto de Contenido tarjeta, vive dentro de cada tarjeta.
- **Checklist:** Value object, igual que la tarea, es un subtipo de Contenido tarjeta. Vive dentro de cada tarjeta y carece de identidad.

7. Agregados

El concepto de agregado se define como un conjunto de objetos del dominio que se tratan como una unidad de consistencia y cuya coherencia se protege mediante reglas del dominio.

Nuestro DDD consta de 3 agregados que cobran sentido, mantienen la coherencia y preservan las reglas del dominio al ser agrupados:

- **Agregado Tablero – Tablero como raíz del agregado →** El primer agregado que consideramos es el de Tablero, el cual representa el espacio de trabajo y contiene las **políticas globales** de las listas y tarjetas y la configuración que afecta al comportamiento del sistema. Es importante recalcar que no contiene directamente las listas ni las tarjetas como parte de su agregado, sino que actúa únicamente como contexto que define las reglas que deben de cumplirse en otros agregados. Es responsable de:
    - Gestionar la URL única de acceso
    - Mantener el estado de bloqueo
    - Configurar reglas globales y por lista
    - Garantizar coherencia entre las políticas de los invariantes

  Las operaciones que implican modificar simultáneamente listas o tarjetas (como mover una tarjeta o eliminar una lista con todas sus tarjetas) se coordinan desde los servicios de aplicación aceptando consistencia eventual entre agregados, condición aceptable para este sistema.

- **Agregado Lista de tareas – Lista como raíz del agregado →** Este agregado representa un espacio dentro del tablero que contiene tarjetas. Cada lista tiene identidad propia y está asociada a un identificador de tablero, pero mantiene su propio límite transaccional. Es responsable de:
    - Mantener el conjunto de las tarjetas contenidas y su orden
    - Aplicar el límite N de tarjetas por lista si está configurado
    - Garantizar que no se exceda la capacidad configurada
    - Gestionar la eliminación en cascada de las tarjetas que contiene cuando se elimina una lista

  Las operaciones que implican coordinación entre listas y tarjetas (como por ejemplo mover una tarjeta entre listas) se gestionan mediante los servicios de aplicación, aceptando de nuevo consistencia eventual entre agregados.

- **Agregado Tarjeta – Tarjeta como raíz del agregado →** Representa la unidad de información dentro de la una lista. Cada tarjeta tiene identidad propia y mantiene un ciclo de vida independiente al resto de agregados. Es responsable de:
    - Gestionar el contenido propio dentro de la tarjeta
    - Mantener las etiquetas asignadas a cada tarjeta

- Registrar la lista actual a la que pertenece y las listas por las que ha pasado para validar prerrequisitos
- Cambiar su estado (moverse entre listas, editar el contenido, eliminar la tarjeta o marcarla como completada, que será en esencia, moverla a una lista especial)

Este agregado Tarjeta no tiene la responsabilidad de validar las reglas globales del tablero. Dichas validaciones se realizan consultando el agregado del Tablero mediante servicios de aplicación. Al separar Tarjeta y considerarlo un agregado independiente se permite mayor concurrencia, se reduce el tamaño del agregado Tablero y favorece la escalabilidad.

- **Agregado Usuario – Usuario como raíz del agregado →** Su identidad y su ciclo de vida es propio e independiente al tablero. Un mismo usuario puede crear o acceder a varios tableros, lo que lo hace independiente. Cada usuario almacena una lista con identificadores de tableros. Además, las reglas de cada tablero del usuario pueden ser diferentes. Se hace referencia a él desde otros agregados mediante el correo electrónico del usuario.
- **Agregado Plantilla – Plantilla como raíz del agregado →** La plantilla es una entidad reutilizable que es **independiente** del tablero. La plantilla se crea una vez y se puede usar para crear **muchos tableros**, con las mismas listas y reglas, pero diferentes en esencia y con ciclos de vida distintos. También sigue sus propias invariantes, como el cumplimiento del esquema correcto del YAML. La coherencia entre las reglas de un tablero y la plantilla serán **heredadas** del propio tablero, pero pasarán a formar parte de las características de la plantilla. Dicho de otra forma, se copia la estructura del tablero pero se desentiende de aquel tablero a partir del cual se ha creado la plantilla.
- **Agregado Entry Historial – Entrada Historial como raíz del agregado →** Este agregado representa una traza inmutable de una acción que ha ocurrido en el tablero. El principal objetivo es permitir la consulta del historial de actividades de cada tablero sin necesidad de cargar el agregado tablero con toda su información. El historial de un tablero se construye como la consulta de todas las Entrada Historial cuyo TableroId coincide con el tablero que se consulta. Se modela como agregado independiente además porque tiene un patrón de acceso distinto: es append-only para la escritura y consulta paginada para la lectura, mientras que el tablero necesita consistencia sobre las listas y las tarjetas.

8. Servicios de aplicación

Los servicios de dominio contienen lógica que no pertenece exclusivamente a una sola entidad o value object o que orquesta varias responsabilidades.

- ServicioGestionTablero → Maneja las operaciones complejas sobre el agregado tablero. Aunque muchos de estos métodos se ejecutan en métodos dentro del propio agregado, este servicio de dominio actúa como fachada entre casos de uso (valida precondiciones de aplicación, inicia la transacción, invoca métodos del agregado, crea las entries del historial…)

```
public class ServicioGestionTablero {

	// TABLERO
	TableroId crearTablero(CrearTableroCmd cmd);
	void renombrarTablero(TableroId id, String nuevoNombre, Actor actor);
	void eliminarTablero(TableroId id, Actor actor);
	void bloquearTablero(TableroId id, EspecBloqueo espec, Actor actor);
	void desbloquearTablero(TableroId id, Actor actor);

	// LISTAS
	ListaId crearLista(TableroId id, String nombre, Actor actor);
	void renombrarLista(TableroId id, ListaId listaId, String nuevoNombre,
Actor actor);
	void eliminarLista(TableroId id, ListaId listaId, Actor actor);
	void definirListaEspecial(TableroId id, ListaId listaId, Actor actor);
	void configurarLimiteLista(TableroId id, ListaId listaId, int limite,
Actor actor);
	void configurarPrerrequisitosLista(TableroId id, ListaId listaId,
List<ListaId> prerrequisitos, Actor actor);

	// TARJETAS
	TarjetaId crearTarjeta(TableroId id, ListaId listaId, ContenidoTarjeta
contenido, Actor actor);
	void editarTarjeta(TableroId id, TarjetaId tarjetaId, ContenidoTarjeta
nuevoContenido, Actor actor);
	void eliminarTarjeta(TableroId id, TarjetaId tarjetaId, Actor actor);
	void moverTarjeta(TableroId id, TarjetaId tarjetaId, ListaId
listaDestino, Actor actor);
	// Realmente es mover una tarjeta a la lista especial
	void completarTarjeta(TableroId id, TarjetaId id, Actor actor);


	// ETIQUETAS
	void addEtiquetaATarjeta(TableroId id, TarjetaId tarjetaId, String
nombre, String color, Actor actor);
    void eliminarEtiquetaDeTarjeta(TableroId id, TarjetaId tarjetaId, String
nombre, String color, Actor actor);
    void modificarEtiquetaEnTarjeta(TableroId id, TarjetaId tarjetaId,
            String nombreAntiguo, String colorAntiguo, String nombreNuevo,
String colorNuevo,
            Actor actor);
}
```

- ServicioPlantilla → Responsable de validar y parsear el fichero YAML, guardar plantillas y crear tableros a partir de plantillas.

```
public class ServicioPlantilla {
	// Valida el esquema
	PlantillaId crearPlantilla(String yaml, Actor actor);

	// Devuelve la estructura que utiliza ServicioGestionTablero para crear
el tablero
	EspecsTablero parsearPlantilla(PlantillaId);
}
```

- ServicioHistorial → Actúa como servicio de aplicación de lectura o fachada, no como lógica pura del dominio.

```
public class ServicioHistorial {
	void append(EntryHistorial entry);
	Page<EntryHistorial> consultarPorTablero(TableroId tableroId, PageRequest
pageRequest);
}
```

- ServicioAutenticacion → Genera y valida los códigos de login y los envía por correo. Es un servicio de consulta.

```
public class ServicioAutenticacion {
	void enviarCodigoLogin(Email email);
	AuthToken verificarCodigoLogin(Email email, Codigo codigo);
}
```

- ServicioFiltradoTarjetas → Realiza la consulta de tarjetas filtrando por etiquetas asignadas a dichas tarjetas. También es un servicio de consulta.

```
public class ServicioFiltradoTarjetas {
	Page<Tarjeta> filtrarPorEtiquetas(TableroId tableroId, List<Etiqueta>
etiquetas, ModoFiltrado modo, PageRequest page);
}
```

9. Servicios de dominio

Los servicios de dominio contienen lógica pura de negocio. Pertenecen en general al modelo de dominio y opera sobre diferentes entidades, value objects y agregados. Deben su existencia a que la lógica no encaja de manera natural dentro de una sola entidad. Para este proyecto, tenemos un servicio de dominio PoliticaTarjetas que valida las reglas de negocio que actúan sobre los elementos del dominio:

```
public class PoliticaTarjetas {
```

```
        // Valida bloqueo, límite y prerrequisitos
        validarCreacion(tablero, listaDestino);

        // Valida bloqueo y límite
        validarMovimiento(tablero, tarjeta, listaDestino);

        // Valida que exista lista especial de completadas
        validarCompletar(tablero, tarjeta);
}
```

No forma parte del DDD pero será necesario documentar:

10. Puertos / Repositorios (Interfaces de salida)

Definiremos estas interfaces en la capa de dominio/aplicación (puertos):

```
package servicios;

/* ------------------------- TABLEROS ------------------------- */

interface RepositorioTableros {
    void guardar(Tablero tablero);
    Optional<Tablero> buscarPorId(TableroId tableroId);

    // Necesario para HA6 (hard delete)
    void eliminarPorId(TableroId tableroId);

    // Útil para "tableros disponibles" (HA6). Si no lo quieres, puedes
quitarlo.
    List<TableroId> listarIdsPorUsuario(Email email);
}

/* ------------------------- PLANTILLAS ------------------------- */

interface RepositorioPlantilla {
    void guardar(Plantilla plantilla);
    Optional<Plantilla> buscarPorId(PlantillaId plantillaId);
}

/* ------------------------- LISTAS ------------------------- */

interface RepositorioListas {
    void guardar(ListaDeTareas lista);
    Optional<ListaDeTareas> buscarPorId(ListaId listaId);

    // Necesario para HA6 (hard delete cascada) y para operaciones por tablero
    List<ListaDeTareas> buscarPorTableroId(TableroId tableroId);

    // Necesario para borrar lista (HB1) y cascada al borrar tablero
    void eliminarPorId(ListaId listaId);

    // Útil para borrar todo por tablero sin cargarlo (hard delete)
    void eliminarPorTableroId(TableroId tableroId);
}
```

```java
/* ------------------------- TARJETAS ------------------------- */

enum ModoFiltradoEtiquetas { AND, OR }

interface RepositorioTarjetas {
    void guardar(Tarjeta tarjeta);
    Optional<Tarjeta> buscarPorId(TarjetaId tarjetaId);

    // Necesario para eliminar todas las tarjetas dentro de una lista (HB1)
    List<Tarjeta> buscarPorListaId(ListaId listaId);

    // Necesario para hard delete cascada al eliminar tablero
    List<Tarjeta> buscarPorTableroId(TableroId tableroId);

    // Borrado directo
    void eliminarPorId(TarjetaId tarjetaId);

    // Borrados masivos (eficientes)
    void eliminarPorListaId(ListaId listaId);
    void eliminarPorTableroId(TableroId tableroId);

    // HG1 – Filtrado por etiquetas (consulta)
    Page<Tarjeta> filtrarPorEtiquetas(
            TableroId tableroId,
            List<Etiqueta> etiquetas,
            ModoFiltradoEtiquetas modo,
            PageRequest pageRequest
    );
}

/* ------------------------- USUARIOS ------------------------- */

interface RepositorioUsuarios {
    void guardar(Usuario usuario);
    Optional<Usuario> buscarPorEmail(Email email);
}

/* ------------------------- HISTORIAL (EntryHistorial) -------------------------
- */

interface RepositorioEntryHistorial {
    void append(EntryHistorial entry);

    Page<EntryHistorial> consultarPorTablero(TableroId tableroId, PageRequest
pageRequest);

    // Necesario para HA6 (hard delete) -> "El historial del tablero desaparece"
    void eliminarPorTableroId(TableroId tableroId);

    // Posibles mejoras:
    /*
    // Opcional: filtros si los expones en REST
    Page<EntryHistorial> consultarPorTableroYTipo(
            TableroId tableroId,
```

```
            TipoEntry tipo,
            PageRequest pageRequest
    );

    // Opcional: si quieres rango de fechas
    Page<EntryHistorial> consultarPorTableroYRango(
            TableroId tableroId,
            Instant from,
            Instant to,
            PageRequest pageRequest
    );
    */
}

/* ------------------------- EMAIL (autenticación) ------------------------- */

interface PuertoEnvioEmail {
    void enviarEmail(MensajeEmail m);
}

/* ------------------------- YAML (plantillas) ------------------------- */

interface PuertoParserYAML {
    YamlCorrecto parse(String yaml) throws PlantillaInvalida;
}

/* ------------------------- CÓDIGOS DE LOGIN ------------------------- */

interface RepositorioCodigosLogin {
    void guardarCodigo(Email email, CodigoLogin codigo, Instant expiraEn);

    Optional<CodigoLogin> buscarCodigoVigente(Email email);

    void invalidarCodigo(Email email);
}
```

Esto nos permitirá implementar los adapters concretos en Spring Boot.

11. Eventos del dominio

Aquí consideramos todos los eventos de dominio importantes y relativos al sistema que vamos a construir. Muchos de ellos generan registros de tipo Entry Historial.

```
public class EventosDominio {
      TableroCreado(tableroId, actorEmail, desdePlantilla?); // ? == opcional
      TableroEditado(tableroId, actorEmail, nombreAntiguo, nombreNuevo);
      TableroEliminado(tableroId, actorEmail);
      TableroBloqueado(tableroId, especBloqueo, actorEmail);
      TableroDesbloqueado(tableroId, actorEmail);

      PlantillaCreada(plantillaId, actorEmail);

      TarjetaCreada(tableroId, tarjetaId, listaId, actorEmail);
```

```
        TarjetaEditada(tableroId, tarjetaId, actorEmail,
                       contenidoAntiguo, contenidoNuevo);
        TarjetaEliminada(tableroId, tarjetaId, listaId, actorEmail);
        TarjetaMovida(tableroId, tarjetaId, fromListaId, toListaId, actorEmail);
        // Realmente es mover una tarjeta a la lista especial
        TarjetaCompletada(tableroId, tarjetaId, actorEmail);

        ListaCreada(tableroId, listaId, actorEmail);
        ListaEditada(tableroId, listaId, actorEmail, nombreAntiguo, nombreNuevo);
        ListaEliminada(tableroId, listaId, actorEmail);
        LimiteListaConfigurado(tableroId, listaId, limite, actorEmail);
        ListaEspecialDefinida(tableroId, listaId, actorEmail);
        // "Una lista define que una tarjeta tiene que haber pasado por otras
listas antes"
        PrerrequisitosListaConfigurados(tableroId, listaId, listasPrevias,
actorEmail);

        EtiquetaAnadidaATarjeta(tableroId, tarjetaId, actorEmail, nombre, color);
        EtiquetaEliminadaDeTarjeta(tableroId, tarjetaId, actorEmail, nombre,
color);
        // Como son VO, realmente consta de una eliminación y una adición
        EtiquetaModificadaEnTarjeta(tableroId, tarjetaId, actorEmail,
                       nombreAntiguo, colorAntiguo, nombreNuevo, colorNuevo);
}
```

12. Contratos REST

Cada caso de uso importante tendrá un servicio de aplicación que será invocado por un controlador REST. En Spring Boot, los controladores serán adaptadores de entrada y los servicios ServicioGestionTablero, ServicioAutenticacion, ServicioHistorial, ServicioPlantilla y ServicioFiltradoTarjetas serán servicios de aplicación que dependen de puertos (repositorios, envío de email, parser YAML, etc).

Convenciones generales

- Identificadores usados en las rutas: tableroId, listaId, tarjetaId, plantillaId, entryId.
- Cada operación que modifica el estado (POST/PUT/DELETE) genera únicamente una EntryHistorial (R5).
- El historial se consulta paginado (R7), con un máximo de 20 entradas por página según HF2.
- El borrado de tablero es hard delete e implica el borrado del historial asociado (HA6).
- Operaciones de consulta (GET) no generan EntryHistorial.

Para su mejor lectura, todos los contratos REST están definidos en la carpeta “Contratos REST” en la carpeta de docs en el repositorio GitHub. No obstante, se definen también aquí:

12.1. Tableros

```
CREAR TABLERO
-------------------------------------------------------
POST /tableros

Body:
{
  "nombre": "Mi tablero",
  "plantillaId": "opcional"
}

Respuesta (201):
{
  "tableroId": "uuid",
  "urlCompartir": "https://.../tableros/{tableroId}"
}

Servicio:
ServicioGestionTablero.crearTablero(cmd);



CONSULTAR TABLERO
-------------------------------------------------------
GET /tableros/{tableroId}

Respuesta(200) (ejemplo):
{
  "tableroId": "uuid",
  "nombre": "Mi tablero",
  "urlCompartir": "https://.../tableros/{tableroId}",
  "bloqueo": { "activo": true, "desde": "…", "hasta": "…" },
  "listaEspecialCompletadasId": "opcional",
```

```
  "listas": [
    {
      "listaId": "uuid",
      "nombre": "TODO",
      "limiteN": 5,
      "prerrequisitos": ["..."]
    }
  ]
}
```

RENOMBRAR TABLERO

PUT /tableros{tableroId}/nombre

Body:

```
{ "nuevoNombre": "Nuevo nombre" }
```

Respuesta (204)

Servicio:

ServicioGestionTablero.renombrarTablero(tableroId, nuevoNombre, actor)

BLOQUEAR TABLERO

POST /tableros/{tableroId}/bloqueo

Body:

```
{ "desde": "yyyy-mm-ddT00:00:00Z", "hasta": "yyyy-mm-ddT00:00:00Z" }
```

DESBLOQUEAR TABLERO

DELETE /tableros/{tableroId}/bloqueo

Respuesta (204)

Servicio:
ServicioGestionTablero.desbloquearTablero(tableroId, actor)

ELIMINAR TABLERO

DELETE /tableros/{tableroId}

Respuesta (204)

Servicio:
ServicioGestionTablero.eliminarTablero(tableroId, actor)

LISTAR TABLEROS DISPONIBLES

GET /tableros

Respuesta (200) (ejemplo):

```
[
  { "tableroId": "uuid", "nombre": "Tablero A" },
  { "tableroId": "uuid", "nombre": "Tablero B" }
]
```

CREAR PLANTILLA DE TABLERO

POST /plantillas

```
Body:
{ "yaml": "..." }

Respuesta (201)(ejemplo):
{ "plantillaId": "uuid" }

Servicio:
ServicioPlantilla.crearPlantilla(yaml, actor)



CREAR TABLERO DESDE PLANTILLA
-------------------------------------------------------
POST /tableros/desde-plantilla/{plantillaId}

Body:
{ "nombre": "Tablero desde plantilla" }

Respuesta (201)(ejemplo):
{
  "tableroId": "uuid",
  "urlCompartir": "https://.../tableros/{tableroId}"
}

Servicios:
ServicioPlantilla.parsearPlantilla(plantillaId)
ServicioPlantilla.parsearPlantilla(plantillaId)
```

12.2. Listas

```
CREAR LISTA
-------------------------------------------------------
POST  /tableros/{tableroId}
```

Body:

```
{ "nombre": "PENDIENTES" }
```

Respuesta (201):

```
{ "listaId": "uuid" }
```

Servicio:

ServicioGestionTablero.crearLista(tableroId, nombre, actor)

RENOMBRAR LISTA

PUT /tableros/{tableroId}/listas/{listaId}/nombre

Body:

```
{ "nuevoNombre": "EN PROCESO" }
```

Respuesta (204)

Servicio:

ServicioGestionTablero.renombrarLista(tableroId, listaId, nuevoNombre, actor)

ELIMINAR LISTA

DELETE /tableros/{tableroId}/listas/{listaId}

Respuesta (204)

Servicio:

Servicio: ServicioGestionTablero.eliminarLista(tableroId, listaId, actor)

CONFIGURAR LÍMITE N

PUT /tableros/{tableroId}/listas/{listaId}/limite

Body:

{ "limiteN": 5 }

Respuesta (204)

Servicio:

ServicioGestionTablero.configurarLimiteLista(tableroId, listaId, limiteN, actor)

CONFIGURAR PRERREQUISITOS DE LISTA

PUT /tableros/{tableroId}/listas/{listaId}/prerrequisitos

Respuesta (204)

Servicio:

ServicioGestionTablero.configurarPrerrequisitosLista(tableroId, listaId, listasPrevias, actor)

DEFINIR LISTA ESPECIAL DE COMPLETADAS

PUT /tableros/{tableroId}/listas/{listaId}/especial-completadas

Respuesta (204)

Servicio:

ServicioGestionTablero.definirListaEspecial(tableroId, listaId, actor)

12.3. Tarjetas

CREAR TARJETA (tarea/checklist)

POST /tableros/{tableroId}/listas/{listaId}/tarjetas

Body (tarea):

```
{
  "tipo": "TAREA",
  "contenido": { "descripcion": "Hacer..." }
}
```

Body (checklist):

```
{
  "tipo": "CHECKLIST",
  "contenido": { "items": ["A", "B", "C"] }
}
```

Respuesta (201)

Servicio:

ServicioGestionTablero.crearTarjeta(tableroId, listaId, contenido, actor)

EDITAR CONTENIDO DE LA TARJETA

PUT /tableros/{tableroId}/tarjetas/{tarjetaId}/contenido

Body:

```
{
  "tipo": "TAREA",
  "contenido": { "descripcion": "Nueva descripción" }
}
```

Respuesta (204)

Servicio:

ServicioGestionTablero.editarTarjeta(tableroId, tarjetaId, nuevoContenido, actor)

ELIMINAR TARJETA

DELETE /tableros/{tableroId}/tarjetas/{tarjetaId}

Respuesta (204)

Servicio:

ServicioGestionTablero.eliminarTarjeta(tableroId, tarjetaId, actor)

MOVER TARJETA

POST /tableros/{tableroId}/tarjetas/{tarjetaId}/mover

Body:

{ "listaDestinoId": "uuid" }

Respuesta (204)

Servicio:

ServicioGestionTablero.moverTarjeta(tableroId, tarjetaId, listaDestinoId, actor)

MARCAR TARJETA COMO COMPLETADA

POST /tableros/{tableroId}/tarjetas/{tarjetaId}/completar

Respuesta (204)

Servicio:

ServicioGestionTablero.completarTarjeta(tableroId, tarjetaId, actor)

AÑADIR ETIQUETA A TARJETA

POST /tableros/{tableroId}/tarjetas/{tarjetaId}/etiquetas

Body:

{ "nombre": "Urgente", "color": "rojo" }

Respuesta (204)

Servicio:

ServicioGestionTablero.addEtiquetaATarjeta(tableroId, tarjetaId, nombre, color, actor)

ELIMINAR ETIQUETA DE TARJETA

DELETE /tableros/{tableroId}/tarjetas/{tarjetaId}/etiquetas

Body:

{ "nombre": "Urgente", "color": "rojo" }

Respuesta (204)

Servicio:

ServicioGestionTablero.eliminarEtiquetaDeTarjeta(tableroId, tarjetaId, nombre, color, actor)

MODIFICAR ETIQUETA EN TARJETA

PUT /tableros/{tableroId}/tarjetas/{tarjetaId}/etiquetas

Body:

```
{
  "antigua": { "nombre": "Urgente", "color": "rojo" },
  "nueva": { "nombre": "Muy urgente", "color": "rojo" }
}
```

Respuesta (204)

Servicio:

ServicioGestionTablero.modificarEtiquetaEnTarjeta(tableroId, tarjetaId, ... , actor)

FILTRADO DE TARJETAS POR ETIQUETAS

GET /tableros/{tableroId}/tarjetas/filtrar?etiquetas=A,B,C&modo=OR&page=0&size=20

Donde etiquetas es una lista de nombres de etiquetas y el modo es AND u OR (por defecto OR)

Respuesta (200)(ejemplo):

```
{
  "contenido": [
    {
      "tarjetaId": "uuid",
      "listaId": "uuid",
      "contenido": { "tipo": "TAREA", "descripcion": "..." },
      "etiquetas": [{ "nombre": "A", "color": "rojo" }],
      "completada": false
    }
```

```
  ],
  "pagina": 0,
  "tamano": 20,
  "totalPaginas": 3,
  "totalElementos": 45
}
```


Servicio:

ServicioFiltradoTarjetas.filtrarPorEtiquetas(tableroId, etiquetas, modo, page)

12.4. Historial

CONSULTAR HISTORIAL DEL TABLERO

GET /tableros/{tableroId}/historial?page=0&size=20

Respuesta (200)(ejemplo):


```
{
  "contenido": [
    {
      "entryId": "uuid",
      "tableroId": "uuid",
      "tipo": "TARJETA_MOVIDA",
      "emailActor": "user@email.com",
      "timestamp": "2026-03-10T12:00:00Z",
      "detalles": { "tarjetaId": "...", "fromListaId": "...", "toListaId": "..."
}
    }
  ],
  "pagina": 0,
  "tamano": 20,
  "totalPaginas": 10,
  "totalElementos": 200
}
```


Servicio:

```
ServicioHistorial.consultarPorTablero(tableroId, pageRequest)
```

12.5. Autenticación

```
SOLICITAR CÓDIGO LOGIN
-------------------------------------------------------
POST /auth/solicitar-codigo

Body:
{ "email": "user@email.com" }

Respuesta (204)

Servicio:
ServicioAutenticacion.enviarCodigoLogin(email)


VERIFICAR CÓDIGO LOGIN
-------------------------------------------------------
POST /auth/verificar-codigo

Body:
{ "email": "user@email.com", "codigo": "123456" }

Respuesta (200)(ejemplo):
{ "token": "jwt-o-token-de-sesion" }

Servicio:
ServicioAutenticacion.verificarCodigoLogin(email, codigo)
```